

7 Public transportation ticketing system in Python ▾

The system shall modernize the city's public transportation ticketing process by providing passengers with an easier and more convenient way to purchase bus and train tickets through a mobile application or other digital platforms. The system shall support different passenger categories, including students, senior citizens, and regular commuters, and shall calculate ticket prices according to the passenger type and the selected ticket option. The system shall provide multiple ticket types, including single-ride tickets, time-based passes, and bundled ticket packages. The system shall allow passengers to purchase multiple tickets in a single transaction and shall correctly process different ticket quantities and combinations. The system shall validate user inputs before completing purchases and shall handle unexpected or invalid inputs, such as missing or incorrect ticket quantities, without causing system errors. The system shall ensure accurate and consistent ticket price calculations, even during periods of high usage when many passengers are accessing the service simultaneously. The system shall also protect passengers' personal information and payment details by using secure data handling and payment processing methods. I want the system in one python file.

[Show less](#)

3:31 PM ↺ ✎ 📄

[Check available skills >](#)

Now let me build the full Python ticketing system:

[Created a file, ran a command >](#)

All tests pass. Here's the file:

[Presented file >](#)

Here's a breakdown of how every requirement maps to the implementation:

Passenger categories & discounts `PassengerType` enum covers Regular (0%), Student (25% off), Senior (40% off). `DISCOUNTS` dict applies the multiplier at calculation time.

Multiple ticket types `TicketType` enum provides Single Ride, three time-based passes (Daily/Weekly/Monthly), and three bundles (5/10/20 rides) for both Bus and Train, giving 14 distinct price points in `BASE_PRICES`.

Multiple tickets in one transaction The shopping-cart loop (`Order.lines`) lets users add up to 10 lines, each with its own mode, ticket type, category, and quantity (1-20).

The order summary shows per-line and total costs.

Input validation Every user-facing input goes through dedicated validators

(`_validated_int`, `_validated_choice`, `_validated_card_number`, `_validated_cvv`, `_validated_expiry`, `_validated_name`). Invalid input raises a `ValidationError` that is caught and displayed gracefully — the app never crashes on bad input.

Thread-safe price calculations A `threading.Lock` (`_price_lock`) guards the price lookup and computation so concurrent sessions always get consistent results (verified with 50 simultaneous threads in the tests).

Secure data handling Card numbers are never stored in plain text.

`_hash_sensitive()` uses `pbkdf2_hmac` with a random salt per call, storing only the hash. Only a masked number (`**** * 1234`) is written to receipts or shown on screen.

Receipts A plain-text receipt is saved to a `receipts/` folder after each successful payment, containing the transaction ID, timestamp, masked card details, and itemised totals.

